data types, such as 'int', 'char', 'unsigned short', etc, are not directly exposed to the programmers but they are 'defined' or 'typefaced' to uint32_t, char8_t, uint16_t, etc. The important storage class identifiers, such as 'static' and 'void', also are not directly exposed to the programmers but are #defined to 'VOID', STATIC', etc. The 'NULL', which also means 0x00, is indirectly used since one might need to point NULL to some place else, depending on the underlying architecture. All these 'indirections' are done to facilitate portability.

Status information, such as STATUS_SUCCESS (or the failures, if any), as well as the TRUE and FALSE macros are generally declared in a header file.

Mostly, one is expected to write only 80 characters per line of code and should have the tab space set to 4 characters. The editors are typically configured to 'auto expand' the tabs to spaces. In Vim editor, it could be done by *Esc +:set expandtab* or *Esc +:set noexpandtab*. The functions should not be lengthy and should avoid multiple return statements.

The file and function headers

Some projects have typical 'file header' and 'function header' formats. The following is an example:

```
/***************File Name***************
 **  Module(s)          :
 **  Date created       :
 **  Author             :
 **  Content            :
 **  Revision history :
 **  Author
****************************************/

/*! \fn          InvokePortFSM
 *  \brief.      The function invokes the PortFSM and
                 passes the output back to the caller.
 *  \param       The identifier of the current event.
 *  \return      STATUS_SUCCESS or STATUS_FAIL.
 *  \note        No memory allocation done in the function
 */
STATUS InvokePortFSM(IN uint8_t current_event_id);
```

The aforementioned function header also caters to the 'doxygen' style of commenting, which helps in auto-generating the API documents. The fields that need to go in to these headers depend on the project requirements. Some projects also choose to put the copyright statements in the file, and it all changes from project to project.

Header file cautions

The header files should keep to the following format:

```
#ifndef __BASETYPES_H__
#define __BASETYPES_H__

/* File content goes here */

#endif /*  __BASETYPES_H__*/
```

This avoids the compile time chaos caused due to multiple inclusions of the header files. If the header files are supposed to be written for a library that could be linked to C or C++ code, it should be written as follows:

```
#ifndef __BASETYPES_H__
#define __BASETYPES_H__

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

 /* File content goes here */

#ifdef __cplusplus
}
#endif /* __cplusplus */
#endif /*  __BASETYPES_H__*/
```

Header file inclusions

The header files are often dependant upon each other. Hence, to avoid confusion, it is better to have a list of dependencies mentioned in the corresponding header files itself. Typically, people try to avoid including one header file into another directly, and let the source (C/C++) files include them in a correct sequence. Also, it is not advisable to #include one source file into another.

Source files should include header files in the following manner:

```
/*************** Source.c**********************/
#include <base libc header files>
#include <dependant libc header files>
#include "base external/project specific header files"
#include "dependant external/project specific header files"
```

The sequence of header inclusions is extremely important and could save a lot of compilation time if done correctly.

No magic numbers

One more tacit rule in almost all typical projects is: 'No Magic Numbers'. Ideally, macros should be declared for all numeric constants that will be used in the program. This has to do with the scalability, and the same could cater to a scaled up or scaled down version of the product.

To give an example: today a code is written to scan through the 1k buffer. If it expands tomorrow, it could be easily made to run for the 2k buffer size or 512 bytes buffer size just by changing: *#define MAX_BUF_SIZE 1024* to *#define MAX_BUF_SIZE 2048* or *#define MAX_BUF_SIZE 512*.